

Speech and language technology (DA 3452)

...

Lecture 7 – Aug 19
N-Gram Language Models

Recap

Language Modelling as Next-Token Prediction

the batsman hit the ___

MORE PLAUSIBLE

ball

Fits the cricket context
very well.

MORE PLAUSIBLE

boundary

Also fits the cricket
context.

LESS PLAUSIBLE

moon

Possible in general, but
unlikely here.

LESS PLAUSIBLE

democracy

Possible in general, but
very unlikely here.

**Predict the
next token**



$$P(w_t \mid w_1, \dots, w_{t-1})$$

**Combine next-token
probabilities to score
a sequence**



$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

What makes exact full-history prediction difficult to estimate from data?

The N-Gram Approximation

Full history

$$P(w_i \mid w_1, \dots, w_{i-1})$$

Keep only
recent history

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \dots, w_{i-1})$$

Unigram

$$P(w_i)$$

Ignore history

Bigram

$$P(w_i \mid w_{i-1})$$

Use one previous token

Trigram

$$P(w_i \mid w_{i-2}, w_{i-1})$$

Use two previous tokens

More history $\rightarrow$ richer context

More history $\rightarrow$ sparser counts

Once we fix a history, how do we estimate its next-token probabilities from the corpus?

Estimating N-Gram Probabilities from Counts

1 CORPUS REMINDER

the batsman hit the ball
the batsman hit the boundary
the bowler hit the stumps
the batsman defended the ball

2 FIXED-HISTORY COUNTS

History: the batsman

Next word	Count
hit	2
defended	1

$C(\text{the batsman}) = 3$

3 NORMALIZED PROBABILITIES

$$\hat{P}(w | h) = \frac{C(h, w)}{C(h)}$$

$$\hat{P}(\text{hit} | \text{the batsman}) = \frac{2}{3}$$

$$\hat{P}(\text{defended} | \text{the batsman}) = \frac{1}{3}$$

For one fixed history, probabilities across continuations must sum to 1.

Why does maximum likelihood lead to this relative-frequency estimate?

Module 2

Predicting Language and Linguistic Labels

N-Gram Language Models (Ch 3, J&M)

Likelihood: Probability of the Observed Data

Before the general formula, interpret likelihood on one concrete history.

Fix the history

h = the batsman

We hold the history fixed.

Observed data

Continuations seen after this history:

hit, hit, defended

Unknown parameters

$$p_h = P(\text{hit} \mid h)$$

$$p_d = P(\text{defended} \mid h)$$

$$\text{Constraint: } p_h + p_d = 1$$

Definition

$$L(\theta; D) = P(D \mid \theta)$$

The data D are fixed; likelihood is viewed as a function of the model parameters.

Concrete likelihood for this history

$$L(p_h, p_d) = P(\text{hit, hit, defended} \mid p_h, p_d) = p_h \times p_h \times p_d = p_h^2 p_d$$

What likelihood is asking

Among all values of p_h and p_d ,
which ones make the observed data
hit, hit, defended most probable?

Likelihood fixes the **observed data** and asks which **model parameters** make those observations most probable.

Why Does MLE Normalize the Counts?

For one fixed history, maximum likelihood leads directly to relative-frequency estimates.

Step 1: Fix one history

Suppose the history is h , and the possible next words are

$$w_1, w_2, \dots, w_k.$$

Let the observed continuation counts be

$$c_1, c_2, \dots, c_k, \text{ with } c_j = C(h, w_j).$$

Let the unknown continuation probabilities be

$$p_j = P(w_j | h), \text{ with constraint } \sum_{j=1}^k p_j = 1.$$

We are estimating a probability distribution over next words after one fixed history.

Step 2: Write the likelihood

$$L(p_1, \dots, p_k) = \prod_{j=1}^k p_j^{c_j}$$

$$\log L = \sum_{j=1}^k c_j \log p_j$$

The log converts a product into a sum, making optimization easier.

Step 3: Maximize under the sum-to-1 constraint

Lagrangian: $J = \sum_{j=1}^k c_j \log p_j + \lambda \left(\sum_{j=1}^k p_j - 1 \right)$

Take derivative and set to zero: $\frac{\partial J}{\partial p_j} = \frac{c_j}{p_j} + \lambda = 0 \Rightarrow p_j = -\frac{c_j}{\lambda}$

Use the constraint:

$$\sum_{j=1}^k p_j = 1 \Rightarrow -\lambda = \sum_{j=1}^k c_j = C(h)$$

The constraint determines the normalization constant.

Result: relative frequency estimate

$$p_j = \frac{c_j}{\sum_{\ell=1}^k c_\ell}$$

In n -gram notation:

$$\hat{P}(w_j | h) = \frac{C(h, w_j)}{C(h)}$$

Interpretation: for one fixed history, each continuation gets a probability equal to its share of the total history count.

Example:

$$\hat{P}(\text{hit} | \text{the batsman}) = \frac{2}{3}$$

$$\hat{P}(\text{defended} | \text{the batsman}) = \frac{1}{3}$$

MLE turns **continuation counts** for one fixed history into a **conditional probability distribution** by normalizing with the **total history count**.

A Real Corpus Example: Berkeley Restaurant Bigrams

A larger corpus from Jurafsky & Martin's restaurant example.

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

Question for the class: What do you notice first when you look at the table?

How to read the table

Row = previous word
Column = next word
Cell = bigram count

What the corpus contains

Many histories
Many possible continuations
Counts learned from real sentences

What we will do next

Pick one row and then
Normalize it into a conditional
probability distribution

A real count table makes the bigram model concrete: **each row is a history**, and each row can be **normalized** into next-word probabilities.

From Bigram Counts to Bigram Probabilities

For a fixed history, divide each continuation count by the total count of that history.

One row from the J&M count table

History word (row): chinese

Next word	i	want	to	eat	chinese	food	lunch	dinner
Count	1	0	0	0	0	82	5	2

Total history count from J&M unigram counts:

$$C(\text{chinese}) = 158$$

Only a few columns are shown here; the full vocabulary is much larger.

Normalize the row

$$P(w \mid \text{chinese}) = \frac{C(\text{chinese}, w)}{C(\text{chinese})}$$

Divide every continuation count in this row by **158**.

Why not divide by the visible row sum?

Because the textbook figure shows only a subset of all possible next words.

Example probabilities

$$P(\text{food} \mid \text{chinese}) = \frac{82}{158} \approx \mathbf{0.52}$$

$$P(i \mid \text{chinese}) = \frac{1}{158} \approx \mathbf{0.0063}$$

$$P(\text{want} \mid \text{chinese}) = \frac{0}{158} = \mathbf{0}$$

	i	want	to	eat	chinese	food
i	0.002	0.33	0	0.0036	0	0
want	0.0022	0	0.66	0.0011	0.0065	0.0065
to	0.00083	0	0.0017	0.28	0.00083	0
eat	0	0	0.0027	0	0.021	0.0027
chinese	0.0063	0	0	0	0	0.52
food	0.014	0	0.014	0	0.00092	0.0037

Normalize by the **count of the history** — not by the sum of the columns that happen to be **displayed**.

From Bigram Counts to Bigram Probabilities

For a fixed history, divide each continuation count by the total count of that history.

One row from the J&M count table

History word (row): chinese

Next word	i	want	to	eat	chinese	food	lunch	dinner
Count	1	0	0	0	0	82	5	2

Total history count from J&M unigram counts:

$$C(\text{chinese}) = 158$$

Only a few columns are shown here; the full vocabulary is much larger.

Normalize the row

$$P(w \mid \text{chinese}) = \frac{C(\text{chinese}, w)}{C(\text{chinese})}$$

Divide every continuation count in this row by **158**.

Why not divide by the visible row sum?

Because the textbook figure shows only a subset of all possible next words.

Example probabilities

$$P(\text{food} \mid \text{chinese}) = \frac{82}{158} \approx \mathbf{0.52}$$

$$P(i \mid \text{chinese}) = \frac{1}{158} \approx \mathbf{0.0063}$$

$$P(\text{want} \mid \text{chinese}) = \frac{0}{158} = \mathbf{0}$$

	i	want	to	eat	chinese	food
i	0.002	0.33	0	0.0036	0	0
want	0.0022	0	0.66	0.0011	0.0065	0.0065
to	0.00083	0	0.0017	0.28	0.00083	0
eat	0	0	0.0027	0	0.021	0.0027
chinese	0.0063	0	0	0	0	0.52
food	0.014	0	0.014	0	0.00092	0.0037

Key idea: the denominator is the full history count $C(\text{chinese})$, not the sum of the displayed columns.

Normalize by the **count of the history** — not by the sum of the columns that happen to be **displayed**.

Sentence Start and Stop

A language model must assign probabilities to the beginning and end of a sequence too.

i want chinese food

<s> i want chinese food </s>

Beginning

$P(i \mid \langle s \rangle)$

Inside the sentence

$P(\text{want} \mid i)$

$P(\text{chinese} \mid \text{want})$

$P(\text{food} \mid \text{chinese})$

Ending

$P(\langle /s \rangle \mid \text{food})$

<s> supplies a history for the first word. </s> lets the model learn when a sentence ends.

For higher-order n-grams, we may use more than one start symbol.

Sentence boundaries are part of the probabilistic model, even though they are **not** ordinary words.

I want Chinese food!

Once every transition has a probability, we can score the complete sequence.

<s> i want chinese food </s>

$P(<s> i want chinese food </s>)$

$= P(i | <s>) \times P(want | i) \times P(chinese | want) \times P(food | chinese) \times P(</s> | food)$

$$= 0.25 \times 0.33 \times 0.0065 \times 0.52 \times 0.68$$
$$\approx 1.90 \times 10^{-4}$$

Each factor is one next-word prediction under the bigram model.

An **n-gram** language model scores a sequence by **multiplying its successive conditional probabilities**.

J&M Worked Example: "I Want English Food"

A complete sentence probability is obtained by multiplying the successive bigram probabilities, including boundary symbols.

<s> i want english food </s>

$P(<s> i \text{ want english food } </s>)$

$= P(i | <s>) \times P(\text{want} | i) \times P(\text{english} | \text{want}) \times P(\text{food} | \text{english}) \times P(</s> | \text{food})$

0.25

×

0.33

×

0.0011

×

0.50

×

$\approx 3.1 \times 10^{-5}$

$= 0.000031$

Each factor is one next-word prediction under the bigram model.

What this shows

A sentence score is built from local bigram probabilities.

Boundary symbols matter too.

One very small factor can strongly reduce the whole sentence probability.

An n-gram language model scores a sequence by multiplying its successive conditional probabilities.

Log Probabilities

Many tiny probabilities create a numerical problem.

Probability space

$$P(W) = p_1 \times p_2 \times p_3 \times p_4 \times \dots \times p_n$$

$$0.25 \times 0.33 \times 0.0065 \times 0.52 \times 0.68 \times \dots$$

Multiplying many numbers below 1 produces an extremely small result.

Log space

$$\log P(W) = \log p_1 + \log p_2 + \log p_3 + \dots + \log p_n$$

$$P(W) = \exp(\log p_1 + \log p_2 + \log p_3 + \dots + \log p_n)$$

Addition in log space is numerically safer than repeated multiplication in probability space.

NOTE : The probabilistic model stays the same; Only the computation changes.

Log probabilities change **how we compute the score**, **not** the probabilistic model being represented.

What Did the Counts Actually Learn?

N-gram statistics capture whatever regularities are present in the corpus.

Linguistic regularities

to → verb
food → sentence end is
plausible

Some patterns reflect **syntax**
and **local language structure**.

Task / domain regularities

<s> → i
want → chinese
(in a restaurant-request
corpus)

Some patterns reflect what
users are trying to do in this task.

Corpus / cultural regularities

chinese may be frequent
english may be rare
requests reflect the
corpus

Some patterns reflect **who produced**
and **what was collected**.

A language model learns statistical regularities
of its training corpus — not language in isolation.

Corpus choice matters because the model learns what the corpus makes frequent, rare, or likely.

The sparsity problem

Even with a real corpus, the bigram count table is dominated by many unseen word pairs.

A tiny view of a bigram count table

		Next word (column)				
		want	to	chinese	food	lunch
Previous word (row)	i	827	0	0	0	6
	want	0	609	0	82	0
	chinese	0	0	0	5	0
	lunch	0	0	2	0	0
	food	0	0	0	0	0

Observation

Many possible bigrams were never seen in the training corpus.

Why this matters

A zero count may reflect limited data, not a linguistic impossibility.

Vocabulary grows, sparsity grows

More words create many more possible word pairs than the corpus can fully cover.

Does zero count really mean impossible in the language?

Count tables are **sparse**: many possible bigrams are **unseen**, even in a real corpus.

Zero Counts Give Zero Probability

With maximum-likelihood estimation, an unseen bigram receives probability 0 — and one zero factor can wipe out an entire sentence probability.

Unseen bigram

If $C(w_{i-1}, w_i) = 0$, then

$$P_{\text{mle}}(w_i | w_{i-1}) = 0$$

No observed count means zero probability under MLE.

Seen bigram

If $C(w_{i-1}, w_i) > 0$, then

$$P_{\text{mle}}(w_i | w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$$

Observed continuations receive nonzero probability.

Effect on a whole sentence

$$P(w_1, \dots, w_n) = \dots \times P(w_t | w_{t-1}) \times \dots$$

If one factor is 0, then the entire product is 0.



Sentence probability = 0

Interpretation problem

A plausible continuation may be unseen simply because the corpus is finite.

What we need next

Reserve some probability mass for unseen but possible continuations.

Zero counts are too brittle for language modelling: **smoothing** is needed so unseen but plausible events are not treated as **impossible**.

How do we know the model is good?

Estimate probabilities on one corpus; evaluate prediction on unseen text.

Training set

Where counts are collected

$C(h, w), C(h)$

want → chinese:	1
chinese → food:	82

Learn / estimate

Development set

Where modelling choices are tried

$n = 2$ or 3 ?

Which smoothing?

Which hyperparameters?

Tune / choose

Test set

Held-out text for final evaluation

How surprised is the model?

Evaluate once

If the test text influences training or tuning, the score becomes artificially optimistic.

Next question: what **numerical score** should compare two language models on the same held-out test set?

Language models are evaluated by how well they predict **held-out data**, not by how well they remember the training corpus. **predictive evaluation**.

Why raw sentence probability is not enough

A longer sequence gets a smaller probability even when the model behaves the same at each step.

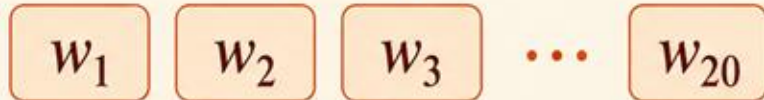
Short sequence



$$P(W_{\text{short}}) = 0.5 \times 0.5 \times 0.5 = \mathbf{0.125}$$

3 predictions

Long sequence



$$P(W_{\text{long}}) = 0.5^{20} \approx \mathbf{9.5 \times 10^{-7}}$$

20 predictions

Lower raw probability does not necessarily mean a worse model; it may only mean a longer sequence.

How can we compare models *per token* rather than *per whole sentence*?

We need a length-normalized score: this leads to perplexity.

Perplexity: probability normalized per token

Raw sequence probability becomes comparable after taking the inverse Nth root.

1. Sequence score

$$P(W) = \prod_i P(w_i \mid \text{history})$$



2. Invert

$$\frac{1}{P(W)}$$



3. Normalize by length

$$PP(W) = P(W)^{-1/N} = \sqrt[N]{\frac{1}{P(W)}}$$

	Raw sequence probability	Length (N)	Perplexity (PP)
Short	$P(W_{\text{short}}) = 0.125$	$N = 3$	$PP = (0.125)^{-1/3} = 2$
Long	$P(W_{\text{long}}) = 0.5^{20}$	$N = 20$	$PP = (0.5^{20})^{-1/20} = 2$

Raw probabilities differ, but perplexities are equal because the **average per-token** probability is the same.

Interpretation

- Lower perplexity = higher average probability on held-out text
- Perfect prediction gives $PP = 1$
- For fair comparison: same test set, same tokenization, same vocabulary convention

Perplexity asks: **on average**, how surprised is the model at **each next token**?